Corrigibility as a Structural Precondition for Digital Public Infrastructure: A Cybernetic Framework

Anivar A Aravind*[1]

[1]Independent Researcher

January 2026

Abstract

Digital Public Infrastructure (DPI) operates at the scale of populations. Systems governing identity, payment, and data exchange effectively constitute a new administrative state. While international definitions from the G20 and United Nations articulate normative aspirations (that systems *should* be open, inclusive, and safe), they largely lack the structural requirements to verify these attributes.

This paper proposes a formal evaluation framework based on **corrigibility**: the structural capacity of those affected by a system to detect error, signal harm, and trigger correction. We synthesize necessary conditions from three traditions: cybernetics (Ashby's Law of Requisite Variety), commons governance (Ostrom's design principles), and free software (Stallman's four freedoms). The resulting framework identifies five non-negotiable properties: **Reversibility, Inspectability, Verification, Constraint, and Reproduction**. We demonstrate that a failure in any single dimension breaks the feedback loop required for system stability. Furthermore, we extend this framework to learned systems (AI) and establish evidentiary standards for distinguishing authentic governance from performative compliance. The framework is released under CC0 public domain.

1 Introduction

Digital Public Infrastructure is not merely a collection of software stacks; it is the encoding of political arrangements into technical artifacts [Winner, 1980]. Access to essential survival services (food distribution, banking, healthcare, mobility) is increasingly mediated through these systems. Because these systems apply fixed rules to the infinite variety of human life, they will inevitably misclassify, exclude, or fail specific users. This is not a malfunction; it is a mathematical certainty when finite rule sets encounter human diversity.

The critical question for a digital polity is not whether errors occur, but whether the architecture permits those affected to **detect, correct, and reverse** them before harm becomes permanent.

Current policy discourse defines DPI through aspirational language. The G20 New Delhi Declaration (2023) and the UN/UNDP framework [UNDP, 2024] describe systems that "should be secure" or "can be built on open standards." These definitions exclude almost nothing. They articulate intent, not condition. Under such broad definitions, a system that traps users in a coercive loop can be labeled "public infrastructure" simply because it operates at scale.

This paper defines corrigibility not as a moral preference, but as a structural property essential for stability. We argue that **unverifiable constraint is structurally indistinguishable from absolute operator control.** Therefore, public legitimacy requires more than policy assertions; it requires a cryptographic chain of custody and a verifiable capacity for correction.

Paper Structure. Section 2 diagnoses why current DPI definitions fail. Section 3 establishes the theoretical foundations from cybernetics, commons governance, and free software. Section 4 derives the five structural tests.

*Corresponding author: anivar.net · ping@anivar.net · ORCID: 0009-0009-8995-0005

Section 5 analyzes temporal dynamics and the correction velocity inequality. Section 6 extends the framework to AI systems. Section 7 provides empirical evaluation of existing infrastructure. Section 8 addresses implications, including essential services and agentic scaling. The appendices formalize the control-theoretic proofs and provide machine-readable schemas for automated assessment.

2 The Problem: Definitions Without Structure

The accepted definitions of Digital Public Infrastructure share a common defect: they describe desired outcomes without specifying the architectural preconditions required to achieve them. The standard formulation (that DPI is a set of "shared digital systems" built on "open standards" [G20, 2023]) leaves open four distinct vectors for capture.

First, **Open Standards are not Open Execution.** A proprietary system can speak a standard language (like a browser displaying HTML) while remaining entirely opaque in its internal logic. The interface is public; the machine is private. (Addressed by the **CODE** test, Section 4.)

Second, **Legal Frameworks do not guarantee Technical Rights.** A system that requires a court order to correct a database error effectively denies correction to the majority of its users. Legal remedy operates on bureaucratic time; infrastructure operates on digital time. (Addressed by the correction velocity inequality, Section 5.)

Third, **Aspiration is not Compliance.** Definitions that rely on what a system "can" do or "should" be offer no resistance to authoritarian drift. Modal language creates no enforceable constraint. (Addressed by the **GOVERN** test, Section 4.)

Fourth, **Scale is not Accountability.** A system that serves a billion users achieves universality, but universal deployment does not confer public legitimacy. Population-scale operation may simply mean population-scale capture. The "public" in such definitions refers to the number of subjects, not the structure of control. (Addressed by the essential services problem, Section 8.)

Without hard structural requirements, "Public" becomes a mere descriptor of scale rather than a mode of accountability. Just as classification systems become invisible as they gain acceptance [Bowker and Star, 1999], incorrigible infrastructure recedes into the background, becoming unchallengeable precisely as it becomes essential. This danger is acute in the transition to AI-mediated governance, where "Infrastructure" begins to make probabilistic judgments rather than deterministic records.

3 Theoretical Foundations

This framework does not invent new ethical categories. Instead, it triangulates a single structural requirement from three independent intellectual traditions. Whether viewed through physics, political economy, or code, the stability of a system depends on the capacity of its subjects to provide corrective feedback.

3.1 Cybernetics and Requisite Variety

In cybernetics, the relationship between a controller and its environment is governed by Ashby's Law of Requisite Variety [Ashby, 1956]. The law states that for a system to be stable, the number of control states must meet or exceed the number of environmental states:

$$V(\text{controller}) \geq V(\text{disturbance}) \tag{1}$$

In DPI, the "disturbance" is the boundless diversity of the human population. No pre-programmed rule set can match this variety. Stability, therefore, relies on feedback channels that transmit error signals from the population back into the system's behavior. If these channels (specifically the ability to exit or correct) are blocked, the controller's variety effectively collapses. The system loses the capacity to regulate its own impact, leading inevitably to volatility or oppression.

3.2 The Commons and Constitutional Constraint

Elinor Ostrom's work on commons governance [Ostrom, 1990] establishes that sustainable systems require monitors who are accountable to the users and collective-choice arrangements that allow users to modify the rules. A system in which rule-making is the exclusive preserve of the operator acts as an enclosure, not a commons.

Critically, Ostrom identifies that governance is not merely about access; it is about graduated sanctions and low-cost conflict resolution. Applied to digital infrastructure, this implies that users must possess **Constitutive Power**: the ability to set binding limits on the system's behavior. If a system's constraints are merely advisory or can be overridden by the operator at will, the system is fundamentally ungoverned.

3.3 Free Software and the Right to Reproduce

The Free Software movement [Stallman, 2002] contributes the mechanical means of enforcement. Stallman's "Four Freedoms" are often misread as a philosophy of sharing, but they are functionally a philosophy of power. The freedom to study (Inspectability) ensures power is visible; the freedoms to modify and distribute (Reproduction) ensure that power is not a monopoly.

This tradition clarifies that openness is not a passive property (reading code) but an active one (forking infrastructure). Transparency without the power to act on it is voyeurism. The ability to reproduce the system independent of its original steward is the only ultimate check against operator capture.

4 The Structural Conditions of Corrigibility

We define corrigibility as the existence of specific mechanical capabilities that allow the subjects of a system to maintain control over it. Digital Public Infrastructure cannot rely on intent or policy to guarantee this property; it requires a specific architecture.

These capabilities form a hierarchy of defense. If a system lacks any single capability, the feedback loop breaks. Such a system becomes structurally indistinguishable from a monopoly exercising absolute control.

4.1 Test 1: EXIT (Reversibility of Participation)

The foundational condition of any corrigible system is the **Reversibility of Participation**. Irrevocable consent functions structurally as conscription. For an infrastructure to operate as a public utility rather than a coercive instrument, users must possess the capacity to withdraw from it without incurring disproportionate penalty or material exclusion.

In practice, many systems operate as a ratchet. They are easy to enter but functional imperatives make them impossible to leave. **Aadhaar** exemplifies this failure: opting out results in exclusion from banking, food rations, and mobile connectivity. When a system becomes a prerequisite for existence, refusal acts as self-immolation rather than feedback. **UPI** partially mitigates this because cash remains legal tender, but network effects increasingly marginalize non-participants. **Linux**, by contrast, imposes no survival penalty: a user can migrate to BSD or Windows without losing access to essential services.

From a cybernetic perspective, EXIT constitutes the primary negative feedback loop. Blocking this channel silences the user's error signal. This blockade overloads the political layer with demands that the technical layer has suppressed.

This dynamic was formalized by Hirschman [Hirschman, 1970] in his analysis of organizational decline. Hirschman distinguished two correction mechanisms: exit (withdrawal from participation) and voice (attempts to change the system from within). His central finding is that these mechanisms exist in productive tension, not as substitutes.

Exit, in Hirschman's formulation, is "neat," "impersonal," and "indirect": one either exits or one does not. This binary quality makes exit a high-bandwidth error signal. When users leave, the system receives unambiguous information that something has failed. Voice, by contrast, is costly and conditioned on the influence and bargaining power that participants can bring to bear. Voice is gradual, political, and requires sustained effort.

Critically, Hirschman demonstrated that exit without voice produces abandonment rather than correction: quality-conscious users flee rather than fight. But voice without exit produces capture: complaints become cosmetic (see Section 5.2.1) because the system faces no credible threat of user departure. The effectiveness of voice depends on the credibility of exit.

This framework illuminates why mandatory enrollment is structurally pathological. By eliminating exit, a system does not merely suppress one feedback channel. It degrades voice itself. When citizens cannot leave, their complaints lack credibility. The operator can absorb infinite grievance without modifying system behavior ($\partial S/\partial G = 0$). Hirschman's insight formalizes what cybernetics predicts: blocking the error signal does not eliminate the error; it eliminates the system's capacity to respond.

4.2 Test 2: CODE (Inspectability of Logic)

Users who cannot leave a system must possess the ability to inspect it. **Inspectability of Logic** requires that the rules determining outcomes are visible to those they govern. Power in a digital system resides in execution. Lessig [Lessig, 2006] established that software architecture functions as regulation: code "determines what people can or cannot do in the first place," unlike legal regulation which sets rules for behavior and retroactively punishes non-compliance. This is regulation without appeal. If the execution path remains hidden, that power is absolute; it is not merely unaccountable but structurally unobservable to those it governs.

Transparency specifications or open standards are insufficient if the executing artifact remains a black box. **Aadhaar's** CIDR (Central Identities Data Repository) is proprietary: the matching algorithm that determines identity cannot be examined. **UPI** publishes API specifications, but the NPCI switching logic remains closed. **Let's Encrypt** demonstrates the alternative: its CA software is fully open source, allowing anyone to audit the certificate issuance logic that secures web traffic. For learned systems, the challenge intensifies. **Llama** publishes inference weights but withholds training data composition and RLHF preferences. This is spec-as-source: you can run the model but cannot understand why it produces specific outputs.

4.3 Test 3: AUDIT (Independent Verification)

While inspection enables understanding, **Independent Verification** enables truth. This functions as the sensor in the cybernetic feedback loop. A system is verifiable only if external, permissionless actors can test its behavior, measure its error rates, and document its failure modes within production environments.

Incorrigible systems typically capture this function. **UPI** permits RBI regulatory audits, but transaction-level verification requires NPCI authorization; independent researchers cannot measure failure rates or dispute resolution times. **Open-weights AI models** (Llama, Qwen, DeepSeek) selectively publish red-teaming results while blocking independent adversarial testing at scale. **Bitcoin** provides the counter-model: every transaction is independently verifiable by any node, and consensus is cryptographic proof. Similarly, **Let's Encrypt** publishes Certificate Transparency logs, making every issued certificate publicly auditable in real time.

4.4 Test 4: GOVERN (Constitutive Constraint)

Verification identifies the error, while **Constitutive Constraint** enforces the correction. This framework distinguishes structural governance from administrative functions like consultation, multi-stakeholder meetings, or feedback forms.

Governance exists only when binding constraints limit system behavior in ways the operator cannot override. **Aadhaar** is governed by UIDAI fiat: the same entity that operates the system sets its rules. **UPI** has NPCI governance, but NPCI is majority-owned by participating banks, creating structural conflicts. **Let's Encrypt** shows what binding constraint looks like: the ISRG charter legally binds the organization to its public benefit mission, with community board representation. **Bitcoin's** BIP process demonstrates governance through proven fork history; contentious changes have been rejected by the network, proving the community can override developer intent. **PostgreSQL's** contributor agreement prevents any single entity from capturing the project.

4.4.1 The Post-Execution Fallacy

A critical failure in modern policy involves relying on exogenous or post-hoc remedies to substitute for structural constraints. Mechanisms such as courts, ombudsmen, and grievance redressal officers operate on "bureaucratic time," measured in months or years. Infrastructure operates on "digital time," measured in milliseconds. A system that executes a harmful action, such as wrongful deletion of a beneficiary, and relies on a court order to reverse it six months later is structurally ungoverned for that duration. True governance must function within the system's execution loop to block prohibited states before they manifest.

Consequently, claims of governance must be evidentiary rather than declarative. They require a signed cryptographic chain of custody linking the operator's authority to an external, non-overrideable instrument.

4.5 Test 5: FORK (Independent Reproduction)

The ultimate check on power is the ability to recreate it. **Independent Reproduction** determines whether the ecosystem allows the infrastructure itself to be separated from its current steward. This requirement distinguishes structural resilience from simple market competition.

The existence of competitors, such as choosing between two proprietary cloud providers, provides an "Exit" option but not a "Fork." Shifting providers allows a user to escape a specific operator, but it forces them to abandon their history, identity, and operational logic. This action is substitution, not reproduction.

A system passes the FORK test only if the community possesses the legal rights, technical artifacts, and economic means to spawn a functionally equivalent instance. **Aadhaar** fails completely: no entity outside UIDAI can reproduce the identity infrastructure. **OpenSearch** demonstrates what successful forking looks like. When Elastic changed its license, AWS forked Elasticsearch and the community continued development independently. **Linux** has been forked thousands of times (Android, Chrome OS, countless distributions). **Llama** presents partial forkability: anyone can run the weights, but reproducing the training requires resources only a handful of companies possess. If Meta abandoned Llama tomorrow, the community could not recreate it.

System	EXIT	FORK	Structural Reality
Cloudflare	PASS (Market)	FAIL	Can switch to Fastly (Exit), cannot reproduce (Proprietary)
AWS	PASS (Market)	FAIL	Can migrate to Azure, logic/state locked (Ecosystem capture)
Aadhaar	FAIL (Mandatory)	FAIL	Cannot refuse (Legal), cannot reproduce (Total capture)
Linux	PASS	PASS	Can leave, can reproduce/fork (Corrigible infrastructure)
Llama	PASS	PARTIAL	Can refuse, can run weights, cannot reproduce training

Table 1: Differentiation between Exit (Substitution) and Fork (Reproduction)

4.6 The Topology of Necessity

Five tests describe the anatomy of a stable control loop rather than a random assortment of normative virtues (see Figure 1).

- **EXIT** provides the Error Signal.
- **AUDIT** acts as the Sensor.
- **CODE** ensures Transmission Clarity.
- **GOVERN** serves as the Actuator.
- **FORK** provides Selection Pressure over the entire loop.

Failure in any single dimension breaks the control loop at a specific physical point. The result is not partial infrastructure but **Open Loop Control**. An open-loop system executes directives without the capacity to sense deviation, interpret error, or apply corrective force.

Under conditions of human variety and environmental change, open-loop control is structurally unstable. Error accumulates without bound. Misalignment compounds. The system drifts toward failure.

Corrigibility is the structural precondition that closes the loop. Only when all five tests are satisfied does a system possess a complete feedback topology capable of detecting divergence, transmitting intelligible signals, executing correction, and replacing failed control logic when necessary. Without this closure, stability is mathematically and operationally impossible.

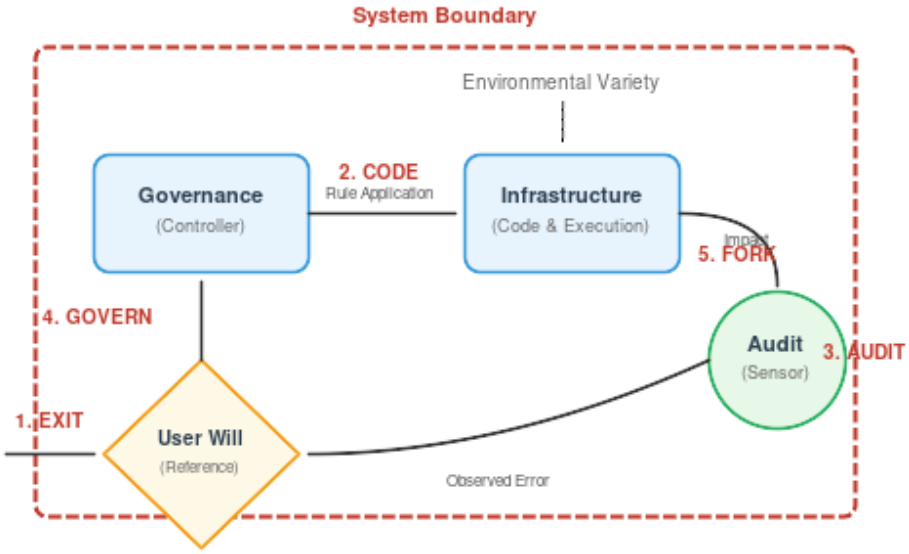

Figure 1: **Topology of Necessity.** Each corrigibility test corresponds to a breakage point in the feedback control loop.

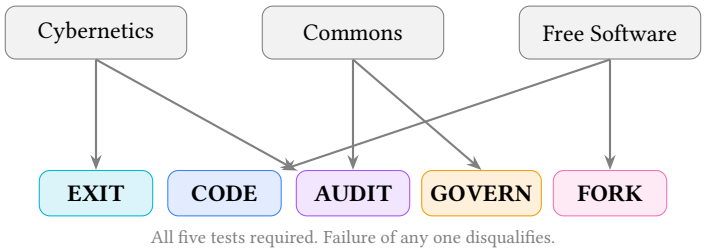

Figure 2: Five corrigibility tests derived from three independent theoretical traditions.

4.7 Structural Symmetry of Power

The five tests form a single power invariant, expressed across layers of control:

Layer	Test	Function (Cybernetic)
Individual	EXIT	Reversibility of participation
Logic	CODE	Inspectability of rules
Logic	AUDIT	Verifiability of error
System	GOVERN	Constraint on controller variance
Ecosystem	FORK	Selection pressure via reproduction

Each test enforces the same principle: *No actor exercising power may be the sole judge of its continuation, scope, or correction.*

A system without EXIT is coercive. A system without GOVERN is arbitrary. A system without FORK is captive.

4.8 Layer-Decomposed Evaluation

Complex infrastructure frequently stratifies into distinct functional layers, each with independent governance, transparency, and reproducibility properties. The five tests must be applied to each layer separately when:

1. Different entities control different layers (protocol versus implementation versus trust framework)

2. Transparency varies across layers (open protocol, closed switching logic)
3. Forkability differs by layer (code is forkable, network effects are not)

4.8.1 Propagation Rule

A system that passes a test at one layer but fails at another does not achieve "partial" compliance. It fails the test. The evaluation proceeds from the layer closest to user impact upward; failure at any layer propagates to the whole.

Formally: Let $L_1, L_2, \ldots, L_n$ be the layers of a system, ordered by proximity to user impact. For test T:

$$T_{\text{system}} = \bigwedge_{i=1}^{n} T(L_i) \tag{2}$$

The system passes T if and only if every layer passes T.

4.8.2 Application to Identity Infrastructure

Layer	Example Components	Typical Failure Mode
Protocol	W3C DID, VC Data Model	Often passes CODE
Implementation	Specific resolver, wallet software	May fail CODE (proprietary)
Credential	Issuer policies, attribute schemas	Often fails GOVERN (unilateral issuer)
Trust	Relying party acceptance, trusted lists	Often fails FORK (cannot reproduce acceptance)

Table 2: Identity infrastructure layer decomposition

The UPI payment rail exhibits similar layering: the protocol specification (NPCI-published) passes CODE at the protocol layer, but the switching logic (NPCI-operated) fails CODE at the implementation layer. Since the implementation layer is closer to user impact, the system fails CODE.

4.8.3 The Weakest-Layer Principle

This propagation rule has a counterintuitive but structurally necessary consequence: a system's corrigibility status equals its weakest layer across any test dimension. Strength at one layer cannot compensate for failure at another.

This principle prevents a common evasion: claiming corrigibility by publishing peripheral artifacts while retaining control over the layers that determine outcomes. Open-washing (Section 7.4) typically exploits layer confusion. Releasing SDKs (passing CODE at the interface layer) while keeping core logic proprietary (failing CODE at the execution layer) does not produce partial corrigibility. It produces total failure at the execution layer, which propagates to the system determination.

5 The Dynamics of Incorrigibility

The five tests define the static architecture of a corrigible system. However, infrastructure exists in time. When we apply these structural requirements to dynamic systems, three fundamental implications emerge regarding stability, energy conservation, and the rule of law. These principles explain why centralized grievance redressal mechanisms—the standard answer of bureaucracies to error—are mathematically insufficient to guarantee stability.

5.1 The Correction Velocity Inequality

Stability in a cybernetic system is not merely a function of whether a correction path *exists*, but of its *latency* relative to error generation. A correction mechanism that is theoretically available (e.g., a court case or a legislative amendment) but slower than the rate of divergence creates a runaway failure state.

We posit the **Temporal Stability Condition**: A system is corrigible only if the rate of effective correction (C) exceeds the rate of error accumulation (E) caused by environmental variety.

$$\frac{d}{dt}C(t) > \frac{d}{dt}E(t) \tag{3}$$

$E(t)$ is driven by environmental volatility—identity fraud tactics, software exploits, pandemics, or economic shocks. These phenomena tend to grow exponentially or appear in high-velocity impulse shocks. In contrast, centralized governance operates on **bureaucratic time**. Legislative changes and judicial reviews are measured in months or years, while technical errors accumulate on **digital time**, measured in milliseconds.

$$\frac{dC_{\text{bureaucratic}}}{dt} \ll \frac{dE_{\text{digital}}}{dt} \tag{4}$$

This timescale mismatch is an instance of the Collingridge Dilemma [Collingridge, 1980]: a double-bind in technology governance. Early in a technology's development, change is easy but the need for change cannot be foreseen (the information problem). Later, when impacts become clear, change has become expensive, difficult, and time-consuming because the technology is entrenched (the power problem).

Digital Public Infrastructure exhibits the Collingridge Dilemma in acute form. By the time exclusion effects are documented (starvation deaths, banking denials, welfare failures), a system may have achieved mandatory status. Correction now requires not merely technical modification but legal amendment, bureaucratic reorganization, and political will. The system's success in achieving scale has made it resistant to correction.

Corrigibility is the structural response to this dilemma: build the correction mechanisms before the need for correction is apparent. EXIT, CODE, AUDIT, GOVERN, and FORK are not remedies applied after failure; they are architectural features that must be present from deployment. A system without these features will eventually require correction but will lack the capacity to receive it.

This inequality explains the failure of the "post-execution" remedies discussed in Section 4.4. Mechanisms like ombudsmen or grievance redressal officers inherently operate at a lower bandwidth and higher latency than the systems they supervise. When the correction loop is thermodynamically too slow to manage the entropy of the environment, error accumulates in the system until it suffers catastrophic collapse. Corrigibility requires closing the gap between error generation and correction; this is only possible if correction logic operates within the technical loop (Test 4), not outside it.

5.2 The Conservation of Correction Demand

When a system fails to correct errors due to the velocity mismatch described above, the demand for correction does not evaporate. It transforms. We propose a principle of **Conservation of Correction Demand**: for a given system state, the total pressure for correction is constant.

$$F_{\text{designed}} + F_{\text{emergent}} = K \tag{5}$$

Here, F_{designed} represents the flow of correction through designed channels—principally **EXIT** (reducing participation) and **GOVERN** (altering rules). When these channels are structurally blocked or throttled by bureaucracy, the pressure shifts to F_{emergent}. Emergent correction channels take the form of protest, litigation, hacks, grey markets, and eventually, civil unrest.

System operators can choose *where* correction occurs; they cannot choose *whether* it occurs. By suppressing EXIT (via mandatory laws) and blocking GOVERN (via executive fiat), authorities force correction demand into channels that are slower, costlier, and fundamentally more destabilizing to the social order. Corrigibility acts as a pressure relief valve that converts potential volatility into system evolution.

5.2.1 Grievance Is Not Feedback

A critical distinction must be drawn between *grievance* and *governance*. Many incorrigible systems act as "roach motels" for complaints: they allow users to submit unlimited feedback (G), but the system state (S) remains invariant to that input.

$$\frac{\partial S}{\partial G} = 0 \tag{6}$$

Grievance channels that record dissatisfaction without a binding mechanism to modify execution, policy, or eligibility logic do not constitute a feedback loop. They serve an aesthetic function, mimicking responsiveness while preserving the controller's rigidity. Under Ashby's Law, if the controller's response variety does not increase in response to signal, regulation has failed.

5.3 The Structural Preconditions of Law

These dynamics have profound implications for constitutional review. Legal doctrines such as proportionality analysis presuppose that state intrusions can be measured, minimized, and remedied. However, modern digital systems frequently violate the structural assumptions upon which these legal doctrines rest.

A court cannot apply proportionality review to a system whose execution is opaque (**CODE**). A legislature cannot remedy harm if the mechanism of harm is an irreversible digital transaction (**EXIT**). A regulator cannot oversee a system if they lack the technical capability to inspect it independent of the operator (**AUDIT**).

Where digital systems block these channels structurally, the rule of law becomes a legal fiction. External review mechanisms cannot function meaningfully if the internal architecture is designed to resist observation and modification. Therefore, **corrigibility** should be understood not merely as a technical specification, but as the *evidentiary precondition* required for the law to take hold of the machine.

6 Extension to Learned Systems

The application of this framework to Artificial Intelligence requires resolving a fundamental crisis of verification. Earlier formulations of the five tests implicitly assumed a deterministic architecture, where the source code is the sole arbiter of behavior, inspection reveals logic, and exact reproduction is mechanically trivial. Learned systems (AI and ML models) dismantle these assumptions. In these systems, identical inputs may yield stochastic outputs; behavior is defined by training data rather than logic gates; and opacity is often an inherent property of the representation rather than a contrivance of the operator.

However, the collapse of deterministic transparency does not invalidate the structural necessity of corrigibility. To the contrary, as the system's internal logic becomes less intelligible to human operators, the external capacity for correction becomes more critical. **The five tests remain invariant; only the evidentiary standard changes.**

6.1 The Invariance of the Standard

Proposition 1. *The five corrigibility tests remain invariant for learned systems. What changes is the verification method.*

The table below illustrates how the mechanical verification of each test adapts to the probabilistic nature of AI without softening the structural requirement.

Test	Deterministic Verification	Learned System Verification
EXIT	Can refuse the system	Disclosure + non-AI alternative
CODE	Source code determines behavior	Layered transparency across training pipeline
AUDIT	Inspect logic and outputs	Statistical bounds + continuous monitoring
GOVERN	Maintainer and RFC process	Governance over objectives and value tradeoffs
FORK	Copy code and deploy	Resource forkability (compute + data)

Table 3: Verification methods for deterministic vs. learned systems

6.2 What Is "Source" in a Learned System?

A central challenge in complying with the **CODE** test is redefining the object of inspection. In a learned system, behavior is distributed across a lineage of distinct artifacts. The inference code is merely the execution engine; the actual behavioral logic is encoded in the weights, which are in turn downstream of the training corpus, filtering decisions, and RLHF preferences.

Therefore, publishing inference code alone does not satisfy the CODE test. It allows an auditor to run the machine, but not to understand why it produces specific outputs. To bridge this gap, compliance requires structured disclosure across the training pipeline, such as **Data Sheets for Datasets** [Gebru et al., 2021] and **Model Cards** [Mitchell et al., 2019]. These artifacts function as the "source code" for structural evaluation; withholding them renders the system a black box.

6.3 The Temporal Variety Gap

Unlike traditional software, which remains static until explicitly patched, learned systems possess a distinct vulnerability related to time. The variety of the controller (V_c) is frozen at the moment of training parameterization (θ), while the variety of the world it operates upon (V_d) continues to expand.

$$\Delta(t) = V(\text{disturbance}; t) - V(\text{controller}; \theta) \tag{7}$$

Without mechanisms for retraining or fine-tuning accessible to the community, the variety gap $\Delta(t)$ widens over time. Thus, AI corrigibility is not a deployment property, but a *persistence condition*.

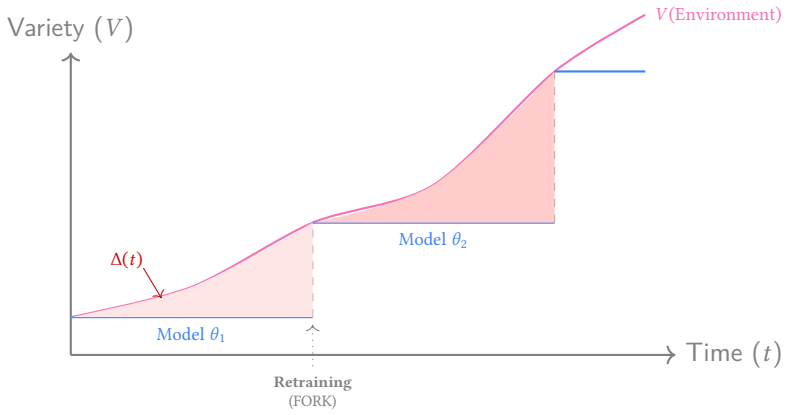

Figure 3: **The Temporal Variety Gap.** Environmental variety $V(e)$ evolves continuously (curve). A fixed model θ (stepped line) diverges from reality. The shaded area represents accumulated error. Only **FORK** rights allow the gap to be closed via retraining.

6.4 Resource Barriers to Forkability

The right to **FORK** a learned system is constrained by physical resources rather than just copyright. In traditional software, the cost to fork is the cost of storage; in AI, the cost to fork is the cost of compute.

Layer	Cost	Barrier
Inference code	Minimal	None
Running open weights	Low	Minor
Fine-tuning	Moderate	Economic
Retraining from scratch	Very high	Structural

Table 4: Resource barriers to forking learned systems

If a community has the legal right to fork a model but lacks access to the original training data or the massive compute clusters required to reproduce it, the "Fork" is illusory.

Claim 1. *Legal permission to fork without access to compute or data is meaningless. This is an infrastructure question, not a licensing question.*

7 Empirical Evaluation

To demonstrate the framework's application, we evaluate systems across three categories: government infrastructure promoted as "DPI," platform infrastructure claiming "openness," and infrastructure that satisfies all five tests.

7.1 Government Infrastructure: The Trap of Partial Compliance

Government-operated systems currently designated as Digital Public Infrastructure frequently exhibit a dangerous failure mode: they achieve partial compliance on technical tests (EXIT, CODE, AUDIT) while failing structurally

on governance tests (GOVERN, FORK). By capturing the monopoly on execution, they render the feedback loop inoperable.

Table 5: **Evaluation of Government Infrastructure.** Centralized execution leads to consistent structural failure despite "open standards" rhetoric.

(a) Aadhaar (India) – Mandatory ID

Test	Result	Evidence
EXIT	FAIL	Mandatory for essential survival services (PDS, Banking) [Khera, 2019].
CODE	FAIL	Proprietary matcher. Execution logic is opaque.
AUDIT	FAIL	Independent failure-rate testing prohibited by law.
GOVERN	FAIL	UIDAI (Operator) acts as Regulator. No citizen veto.
FORK	FAIL	Total centralized capture; cannot be reproduced.

(b) UPI (India) – Payment Rail

EXIT	PARTIAL	Cash exists but network effects coerce adoption.
CODE	PARTIAL	Open protocol specs; Closed switching logic.
AUDIT	PARTIAL	Regulator audits allowed; Public verification blocked.
GOVERN	FAIL	Bank-controlled governance (NPCI). No user seat.
FORK	FAIL	Hub-and-spoke monopoly cannot be forked.

(c) Pix (Brazil) – Central Bank Payment

EXIT	PARTIAL	Other payment methods exist; increasing state reliance.
CODE	PARTIAL	Protocol public; Central switch proprietary.
AUDIT	PARTIAL	Volume data public; Granular error data private.
GOVERN	FAIL	Unilateral Central Bank governance.
FORK	FAIL	Cannot be reproduced independently of BCB.

Pattern: All three systems fail GOVERN and FORK. Partial compliance on technical tests does not satisfy the binary threshold.

7.2 Platform Infrastructure: The Illusion of Openness

In the private sector, systems often leverage "open source" branding while retaining centralized control. This decoupling of artifact transparency from governance reality creates a distinct failure pattern: the artifacts are public; the decisions are not.

Pattern: Open artifacts with closed governance. "Open weights" ≠ "open." "Open source" ≠ "accountable."

7.3 Corrigible Infrastructure

For contrast, Table 7 summarizes seventeen systems that pass all five tests.

Fork as Correction: OpenSearch, Valkey, LibreOffice, and MariaDB demonstrate Ashby's Law in practice. Each emerged when governance variety collapsed: Oracle's Sun acquisition (2010) triggered LibreOffice and MariaDB; unilateral license changes triggered OpenSearch (2021) and Valkey (2024). The ecosystem restored requisite variety through forking.

The framework is governance-agnostic: corporate participation is not disqualifying. Kubernetes (CNCF), Hyperledger, and Let's Encrypt all have significant corporate involvement while passing all tests. The test is whether governance permits correction, not who participates.

License vs. Governance: Redis Ltd. added AGPLv3 in 2025, restoring open licensing. But license is orthogonal to governance. *License* determines what you may do with code; *governance* determines who decides what it becomes. Redis passes one; Valkey passes both.

Table 6: **Evaluation of Private/Platform Infrastructure.** Technical openness (Source/Weights) does not guarantee structural accountability.

(a) Open-Weights AI (Llama, etc.)

Test	Result	Evidence
EXIT	PASS	No vendor lock-in; can run locally.
CODE	PARTIAL	Weights public; Training Data/RLHF opaque.
AUDIT	PARTIAL	Black-box testing possible; Lineage unverifiable.
GOVERN	FAIL	Safety/Utility trade-offs set unilaterally by firm.
FORK	PARTIAL	Fine-tune possible; Retraining blocked by resource cost.

(b) Android (AOSP + GMS)

EXIT	PARTIAL	Ecosystem lock-in high despite device portability.
CODE	PARTIAL	OS open source; Identity/Push services proprietary.
AUDIT	PARTIAL	OS auditable; Google Services opaque.
GOVERN	FAIL	Roadmap dictated by Google.
FORK	PARTIAL	LineageOS possible; App ecosystem breaks without GMS.

(c) Signal Protocol

EXIT	PASS	No lock-in.
CODE	PASS	Fully open source (Client/Server).
AUDIT	PASS	Cryptographically verifiable.
GOVERN	FAIL	Foundation acts unilaterally.
FORK	FAIL	Code forkable; Network locked (no federation).

7.4 The Taxonomy of Open-Washing

Systems that fail these structural tests frequently employ vocabulary to obscure their closure:

1. **Standard-Washing**: Adopting open technical standards (like ISO/W3C) as proxy for governance accountability.
2. **Open-Washing**: Releasing peripheral SDKs while keeping core execution logic proprietary.
3. **Multilateral-Washing**: Using international endorsements (G20, WB) to substitute for technical auditability.
4. **Safety-Washing**: Justifying opacity by claiming inspection poses a security risk.
5. **Consent-Washing**: Obtaining formal consent under conditions of economic coercion.
6. **DID-Washing**: Deploying decentralized identifiers without decentralized governance (see Section 7.4.1).

7.4.1 DID-Washing: The Layer Confusion Problem

Decentralized Identifiers (DIDs) present a distinct challenge to corrigibility evaluation because they separate technical decentralization from governance decentralization across multiple architectural layers. The W3C DID Core 1.0 specification [W3C, 2022] defines DIDs as "globally unique identifiers" that "do not require a centralized registration authority." This technical claim is accurate. However, a system can implement DIDs at the identifier layer while retaining centralized control at every layer that determines actual utility.

Identity infrastructure comprises at least four functional layers, each with independent governance:

A DID that resolves correctly but is not accepted by any relying party is functionally equivalent to no identity at all. The identifier is "self-sovereign"; the identity utility is not.

Empirical Examples. The EU Digital Identity Wallet (EUDI) under eIDAS 2.0 [European Union, 2024] illustrates sophisticated DID-washing. The architecture employs selective disclosure credentials and cryptographic proofs superficially similar to DID/VC ecosystems. However:

- Credential issuance for Person Identification Data is an exclusive government function.

System	Steward	Score	Why Corrigible
Linux Kernel	Linux Foundation	**5/5**	Fully reproducible. Full triad (legal, technical, economic) exists. Independent distributions prove forkability.
Let's Encrypt	ISRG	**5/5**	Open source CA, IETF standards, nonprofit 501(c)(3)
Wikipedia	Wikimedia Foundation	**5/5**	CC BY-SA content, public edit history, community RfC governance
Matrix Protocol	Matrix.org Foundation	**5/5**	Apache 2.0, federated by design, self-hostable
Bluesky (AT Protocol)	Bluesky PBC	**5/5**	MIT/Apache, portable DID identity, self-hostable PDS
PostgreSQL	PGDG	**5/5**	BSD-like license, public mailing lists, major forks exist
IPFS	Protocol Labs	**5/5**	MIT/Apache, public IPIP process, no token governance
Bitcoin	Decentralized	**5/5**	MIT source, public BIP process, proven forks (BCH, BSV)
Kubernetes	CNCF	**5/5**	Apache 2.0, KEP process, distributions (OpenShift, k3s)
Firefox	Mozilla Foundation	**5/5**	MPL 2.0, nonprofit manifesto, forks (LibreWolf, Tor Browser)
Apache HTTP	Apache Foundation	**5/5**	Apache 2.0 license, ASF governance, powers 30% of web
Apache Kafka	Apache Foundation	**5/5**	Apache 2.0, KIP process public, Fortune 500 standard
OpenSearch	Linux Foundation	**5/5**	Forked from Elasticsearch; Apache 2.0, multi-vendor governance
Valkey	Linux Foundation	**5/5**	Forked from Redis; BSD license, community governance restored
Hyperledger	LF Decentralized Trust	**5/5**	Apache 2.0 framework; institutional deployments require separate evaluation
LibreOffice	Document Foundation	**5/5**	Forked from OpenOffice; LGPL/MPL, nonprofit governance
MariaDB	MariaDB Foundation	**5/5**	Forked from MySQL; GPL, foundation governance
Eclipse IDE	Eclipse Foundation	**5/5**	EPL 2.0, 386 member orgs, committer-led governance

Table 7: Infrastructure systems satisfying all five corrigibility tests. All systems demonstrate full EXIT, CODE, AUDIT, GOVERN, and FORK compliance.

Layer	Function	DID-Washing Failure Mode
Identifier	Generate and resolve DIDs	Often decentralized, but meaningless alone
Credential	Issue verifiable claims about the subject	Usually centralized: single issuer dominates
Trust	Determine which credentials are accepted	Controlled by policy, trusted lists, or law
Revocation	Invalidate credentials or identifiers	Often requires issuer cooperation

Table 8: DID-washing failure modes by layer

- Trust determination requires registration in Member State Trusted Lists.
- Wallet certification depends on government-approved Qualified Trust Service Providers (QTSPs).
- Trusted Execution Environment dependency hands cryptographic key governance to mobile OS vendors (Google, Apple).

The result: users control their container (the wallet) but not their capability (the credentials). A journalist critical of their government holds a "decentralized" identifier that can be rendered useless by a single administrative act: revocation from the trusted list.

Implications for Framework Application. DID-washing exploits a gap in naive corrigibility evaluation. A system may appear to pass CODE (the DID spec is public) and FORK (anyone can implement the spec) while failing at the trust layer where actual power resides.

The framework must be applied layer by layer (see Section 4.8). For identity systems:

- CODE must be evaluated at each layer: identifier resolution, credential issuance logic, trust framework rules, and revocation mechanisms.
- GOVERN must ask: who controls the acceptance policy? If a single authority determines which credentials are valid, governance is centralized regardless of identifier architecture.
- FORK must ask: can a community reproduce the trust framework, not merely the identifier scheme? If relying parties are legally required to accept only government-issued credentials, forkability is illusory.

A system passes the DID-washing test only if decentralization extends through the full trust stack: issuance, verification, acceptance policy, and revocation governance. Technical decentralization at the identifier layer with governance centralization at the trust layer is structural theater.

Claim 2. *Identifier decentralization is not system decentralization. The corrigibility of an identity system is determined by its weakest layer across any test dimension.*

	EXIT	CODE	AUDIT	GOV	FORK	Result
Aadhaar	×	×	×	×	×	0/5
UPI	~	~	~	×	×	0/5
Pix	~	~	~	×	×	0/5
Llama etc.	✓	~	~	×	~	1/5
Linux	✓	✓	✓	✓	✓	5/5
Let's Enc.	✓	✓	✓	✓	✓	5/5
K8s	✓	✓	✓	✓	✓	5/5
Bitcoin	✓	✓	✓	✓	✓	5/5
+15 more	See Table 7					

✓= Pass ~ = Partial × = Fail

Figure 4: Comparative evaluation summary. Systems labeled "DPI" (above line) fail structural tests. Corrigible infrastructure (below line) passes all five. See Table 7 for complete list.

7.5 Implications

The contrast is instructive. Aadhaar and UPI, promoted as model DPI, fail all tests. The seventeen systems in Table 7, rarely described as "DPI," pass all tests.

This suggests the "public" in Digital Public Infrastructure often refers to *population-scale deployment* rather than *public accountability*. The framework distinguishes these: scale is not governance.

All systems that pass share common properties:

- Open source with permissive or copyleft licensing
- Nonprofit or community governance structures
- Protocol standardization through open processes (IETF, LKML)

- No monopoly on essential services

Scale does not require capture. Universality does not require opacity. The choice is architectural.

8 Discussion

8.1 The Cybernetics of Binary Evaluation

Critics often argue that this framework's binary pass/fail topology is too rigid for the complex reality of governance, proposing instead "maturity models" or graded scorecards. From a cybernetic perspective, we reject this gradation. Corrigibility is not a variable virtue; it is a **structural connectivity state**.

In a control system, a feedback loop is either closed or it is open. There is no such thing as a "mostly closed" loop. A wire that is disconnected by 1 millimeter transmits zero signal.

- A system that allows **AUDIT** (Sensing) but blocks **GOVERN** (Actuation) is an **Open Loop**. It observes errors but cannot fix them.
- A system that allows **GOVERN** but blocks **EXIT** is a **Coercive Loop**. It traps energy inside the system until it fails catastrophically.

Partial compliance creates "zombie infrastructure"—systems that maintain the aesthetic of responsiveness while severing the actual capacity for stability. Therefore, a "Partial" score in this framework represents a **Fatal Structural Failure**. A system is either corrigible, or it is not.

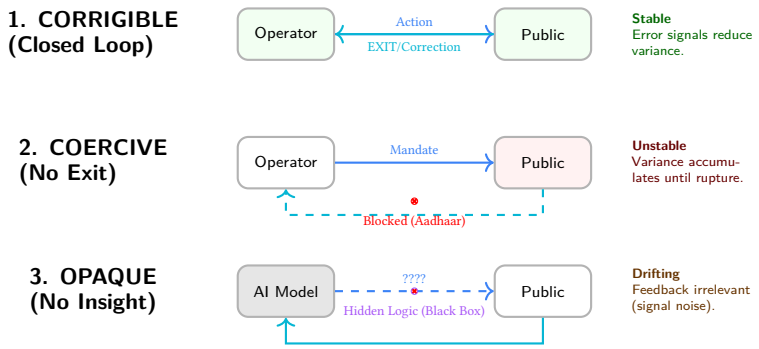

Figure 5: **Topologies of Failure.** (1) A corrigible system closes the loop, converting error signals into stability. (2) A coercive system blocks **EXIT**, trapping energy. (3) An opaque system obscures the action channel, making feedback unintelligible.

8.2 The Protocol Model of State

A common practical objection holds that governments cannot adopt corrigible infrastructure because they must retain control over sovereign functions. This objection relies on a category error: it conflates the *Platform Model* with the *Protocol Model*.

In the **Platform Model** (currently dominant), the state contracts a vendor to build a monolithic silo (e.g., existing digital ID systems). The vendor controls execution, the state attempts to control policy, and citizens are reduced to rows in a database. As Cohen [2019] argues, this model inevitably subverts the Rule of Law by replacing legal due process with optimized platform logic. In the **Protocol Model** (the corrigible alternative), the state adopts open, corrigible protocols and acts as a high-authority participant within them. The state issues credentials and verifies claims, but it does not enclose the infrastructure itself.

This distinction mirrors the architecture of the web. Governments do not need to build their own proprietary browsers to deliver services; they build sites on HTTP. Similarly, corrigible DPI implies that the state becomes an issuer of trust rather than a landlord of infrastructure. This model does not reduce state capacity; it enhances trust by making the state's operations verifiable.

8.2.1 Distributed Assessment Capacity

A corollary objection concerns resources: "Who audits?" The framework does not require every citizen to audit source code. It requires that the infrastructure allows *anyone* to audit, thereby enabling civil society, journalists, and specialized firms to perform this function on behalf of the public. Assessment capacity is itself infrastructure. When systems are designed to be inscrutable, this immune system of society withers.

8.2.2 Structural Preconditions of Law

Legal doctrines such as proportionality analysis—as established in *Puttaswamy v. Union of India* [Supreme Court of India, 2017]—presuppose that state intrusions can be measured, minimized, and remedied. However, as Bhatia [2019] argues, when the architecture of a system is opaque or irreversible, these constitutional guarantees become legal fictions. **Corrigibility** is the evidentiary precondition required for the law to take hold of the machine.

8.3 Corrigibility in Extremis

Incorrigibility is often justified by the rhetoric of necessity—national security, anti-corruption, or emergency efficiency. This framework explicitly denies such exemptions. Corrigibility does not imply that substantive rules are negotiable; a system may enforce strict, non-negotiable limits (e.g., pandemic travel restrictions or environmental caps). However, the *structural capacity* to detect error, verify correct enforcement, and reverse wrongful harm must remain intact even in high-constraint environments. As safety engineering demonstrates, accidents in complex systems are caused precisely by the lack of appropriate constraints on component interactions [Leveson, 2011].

Claim 3. *No exemption from the five tests is granted on the basis of emergency conditions, planetary constraints, or decentralized architecture.*

This applies equally to decentralized systems. Cryptographic immutability is not a substitute for governance. A blockchain that permanently records a theft without a mechanism for consensus-based reversion is not "trustless"; it is strictly incorrigible. If a system allows for irrevocable harm without recourse, its decentralization is merely a distribution of unaccountability.

8.4 Agentic Scaling and Governance Denial of Service

Most contemporary AI systems fail the structural conditions for DPI. While they may satisfy **EXIT** and **CODE** (via open weights), they systematically fail **GOVERN** and **FORK** due to the closure of training data.

We distinguish this from "AI Alignment." Alignment asks: *Can the operator control the system?* Corrigibility asks: *Can the affected subject correct the system?* Structural legitimacy requires the latter.

8.4.1 Corrigibility as an API Contract for Agents

As DPI transitions to support the "Agentic Economy," corrigibility moves from a moral preference to an engineering requirement. Autonomous agents lack the biological resilience to handle bureaucracy.

- **EXIT is Exception Handling**: To an agent, a mandatory system with no exit path appears as a logic deadlock.
- **CODE is Documentation**: Agents require inspectability of logic to form valid plans.
- **AUDIT is Observability**: An optimization agent cannot function without a reliable failure signal.
- **GOVERN is Constraint Discovery**: Advisory guidelines result in undefined behavior for software agents.

Claim 4. *Incorrigible infrastructure is structurally incompatible with AI automation.*

8.4.2 The Human Friction Buffer

Incorrigible infrastructures may appear stable under human load because humans absorb friction through delay, compliance, and informal workaround. Autonomous agents remove this buffering layer. As agentic interaction scales, centralized governance mechanisms face **Governance Denial of Service (GDoS)**: the volume, speed, and strategic diversity of agent actions exceeds the system's capacity to audit, interpret, and correct behavior.

This is not a security failure but a control-theoretic inevitability. Systems lacking distributed audit and rapid, binding governance lack the requisite variety to remain stable under agentic scaling. The correction velocity inequality

(Section 5.1) predicts precisely this failure mode: when $\frac{dE}{dt} \gg \frac{dC}{dt}$, the system diverges. Agents accelerate $E(t)$; centralized governance cannot accelerate $C(t)$.

Claim 5. *Agentic scaling amplifies the correction velocity inequality. Systems that are marginally stable under human interaction rates become catastrophically unstable under agent interaction rates. The human friction buffer is not a feature; it is a hidden subsidy that masks architectural incorrigibility.*

8.5 The Essential Services Problem

A pattern emerges from Section 7: systems that pass all five tests are disproportionately non-essential infrastructure. Linux, PostgreSQL, Kubernetes, and Let's Encrypt power critical operations, but no individual's survival depends on accessing them. The systems promoted as DPI, which mediate access to food, banking, healthcare, and mobility, consistently fail.

This correlation reflects a structural tension between essentiality and corrigibility.

8.5.1 The Essentiality Trap

Essential services create the economic coercion that undermines EXIT. When refusal means exclusion from survival, the feedback loop breaks regardless of other properties. Hirschman [1970] observed that exit is effective precisely because it is voluntary. The credible threat of departure disciplines the system. When departure means death (literal or social), exit ceases to function as feedback. It becomes self-exclusion.

The systems in Table 7 avoid this trap because alternatives exist. A developer who dislikes the Linux kernel's direction can migrate to BSD without losing the ability to compute. A journalist who distrusts Let's Encrypt can obtain certificates elsewhere. The existence of functional alternatives preserves the error signal.

Aadhaar has been made prerequisite to existence. Documented cases demonstrate the consequence: at least 27 starvation deaths in India between 2015 and 2018 were attributed to Aadhaar-related exclusion from food rations [Khera, 2017]. These deaths are not anomalies; they are the structural prediction of a system that blocks EXIT while governing essential services.

8.5.2 Architectural Responses

The tension admits two resolutions, both architectural:

Protected Alternatives. Essential services can remain corrigible if law guarantees functional non-digital equivalents. Cash preserves UPI's partial EXIT; eliminating cash would collapse UPI to Aadhaar's structural status. Corrigibility for essential services requires that non-digital pathways remain legally protected, not merely tolerated.

This is the lesson of the UPI/Aadhaar contrast. Both are Indian government systems operating at population scale. UPI achieves partial EXIT because cash exists; Aadhaar achieves zero EXIT because alternatives have been legally foreclosed. The difference is not in the systems' design but in the ecosystem constraint: whether alternatives are permitted.

Federated Implementation. Rather than a single provider of essential infrastructure, services could be delivered through multiple independent implementations of open protocols. This is analogous to email (SMTP) rather than messaging (proprietary platforms).

Under this model, no single implementation would be essential; the protocol would be. A citizen excluded from one provider could obtain equivalent service from another. Estonian X-Road approaches this architecture: the protocol is open, implementations are distributed across institutions, and the Nordic Institute for Interoperability Solutions (NIIS) provides multi-stakeholder governance. Citizens cannot opt out of the data exchange layer, but no single institution controls their access to all services.

X-Road demonstrates that scale and corrigibility are compatible. It passes CODE (MIT-licensed), AUDIT (transaction logs, public enforcement against misuse), GOVERN (multi-national NIIS), and FORK (deployed in 25+ countries). It fails full EXIT but compensates through strong performance on the other four tests. This compensation is unstable (it depends on continued good-faith governance) but it proves that population-scale infrastructure need not exhibit the total capture of Aadhaar.

8.5.3 The Finding

The absence of fully corrigible essential services is not a limitation of this framework; it is the framework's central finding. The question "Can essential DPI be fully corrigible?" is empirically answered: not under architectures that eliminate alternatives.

8.6 Transition Paths

The framework diagnoses incorrigibility but does not prescribe political remedies. However, the structure of the five tests implies an ordering for transition: certain tests are prerequisites for others, and certain interventions have higher leverage.

8.6.1 Ordered Priorities

The tests form a dependency chain when considered as a transition sequence:

1. **EXIT first.** Restore exit before other tests. Without exit, no other reform creates genuine feedback. This requires either eliminating mandatory linkage laws or guaranteeing functional non-digital alternatives. EXIT is the error signal; without it, the system cannot sense its own failures.
2. **CODE before AUDIT.** Transparency must precede verification. Independent testing is meaningful only if the testing target is observable. Publishing audit results for a black box proves nothing about the box.
3. **AUDIT before GOVERN.** Verified error data creates political pressure for governance reform. Governance without measurement is arbitrary. There is no basis for determining whether constraints are adequate. The sensor must function before the actuator can be calibrated.
4. **GOVERN enables FORK.** Structural governance can mandate open licensing, data portability, and interoperability requirements that enable eventual forkability. GOVERN without FORK leaves the system captive to its current steward; FORK without GOVERN may produce fragmentation without accountability.

This sequence reflects the causal structure of the feedback loop: signal, sensing, transmission, actuation, selection. Interventions at later stages are ineffective if earlier stages are broken.

8.6.2 Phase Transitions

The transition is not gradual improvement. It is a series of phase transitions. Each test either passes or fails; the system remains incorrigible until all five pass. A system that achieves CODE but not EXIT has become transparent without becoming accountable. Users can see their oppression; they cannot escape it.

This explains why open-washing is effective: partial progress creates the appearance of reform while preserving structural capture. The framework rejects partial credit precisely because partial feedback loops do not partially function.

8.6.3 Political Economy

The political economy of transition varies by test:

Table 9: Political economy of transition by test

Test	Transition Barrier	Concentrated Opposition
EXIT	Mandatory linkage laws	Operators lose captive users
CODE	Intellectual property claims	Vendors lose competitive moats
AUDIT	Security-through-obscurity claims	Operators lose narrative control
GOVERN	Sovereignty objections	Executives lose discretion
FORK	Typically least opposed	Requires only licensing and documentation

FORK is typically the least opposed transition because it requires only legal permission and technical documentation. It does not immediately threaten operator control. However, FORK without the prior four tests is meaningless: the right to reproduce a system you cannot inspect, verify, govern, or exit provides no actual power.

8.6.4 Cost Considerations

Transition is not costless. The European Commission [European Commission, 2021] estimated that public sector open-source adoption yields cost-benefit ratios above 1:4, but this ratio assumes successful implementation. Transition costs include personnel retraining, system compatibility engineering, procurement policy updates, and organizational change.

However, these costs must be weighed against the costs of continued incorrigibility:

- Litigation costs from unresolvable grievances
- Political instability from suppressed correction demand (Section 5.1)
- Technical debt from opacity-enabled shortcuts
- Vendor lock-in premiums from non-forkable dependencies
- Human costs: documented deaths, exclusions, and deprivations

The systems in Table 7 demonstrate that corrigibility at scale is economically sustainable. Linux, Kubernetes, and Let's Encrypt operate at population scale with resource models that have proven durable over decades.

9 Conclusion

Digital Public Infrastructure is not a specific technology; it is a claim of political legitimacy. It asserts that a system is sufficiently foundational, accessible, and accountable to serve as the substrate of civic life.

This paper argues that such legitimacy cannot be derived from self-declarations, "open standards" labeling, or multilateral endorsements. It relies on a specific cybernetic property: **Corrigibility**. The capacity of the governed to observe the rules, verify their execution, limit their reach, and, if necessary, reproduce the infrastructure to escape its capture.

We have proposed five rigorous tests to verify this capacity. We have demonstrated that systems currently touted as DPI models—Aadhaar, UPI, and proprietary AI—fail these tests, revealing them to be administrative enclosures rather than public goods. Conversely, we have shown that the systems which actually run the world—Linux, Kubernetes, the Web—pass them structurally.

The choice is not between chaos and control. It is between **Open Loop** systems that drift toward fragility and authoritarianism, and **Closed Loop** systems that stabilize themselves through the correction of those they serve.

> *Systems that cannot be corrected by those they affect will eventually be corrected by forces they cannot control.*

9.1 Limitations

This framework has the following limitations:

1. **Binary classification loses diagnostic information.** A system failing one test is treated identically to one failing all five for the purpose of the corrigibility determination. This is structurally correct (a broken feedback loop is broken regardless of where it breaks) but diagnostically impoverished. The schema (Appendix C) preserves gradation in fields like `penalty_ratio` and `visibility`, but the final determination collapses to binary.
2. **Governance evaluation is judgment-dependent.** CODE and FORK have clear verification criteria for code artifacts. GOVERN is harder to verify: governance claims require historical evidence of enforcement, and such evidence may be contested or incomplete. The evidentiary standards in Appendix D address this, but governance evaluation remains more interpretation-dependent than technical evaluation.
3. **No fully corrigible essential services are identified.** The systems in Table 7 are non-essential infrastructure. Estonian X-Road approaches corrigibility but fails full EXIT. The framework may describe a category that is structurally empty for essential services under current political economy. If so, this is a finding, not a flaw, but it limits immediate prescriptive utility.
4. **Resource forkability for AI remains unresolved.** Section 6.4 identifies that legal rights without compute access may be permanently unfulfillable for large AI systems. The framework diagnoses this as a structural barrier to FORK but does not resolve the underlying resource asymmetry. Whether "compute as infrastructure" could make AI forkability meaningful is an open question.

5. **The timescale argument applies to external remedies.** Section 5.1 argues that bureaucratic correction is structurally too slow for digital-time errors. However, some corrigible systems (Bitcoin's BIP process, Linux's kernel mailing list) operate on human timescales and are accepted as valid governance. The distinction is that these mechanisms are internal to the technical community, not external bureaucracies. The timescale critique applies to external remedies (courts, ombudsmen), not to all deliberative processes.
6. **Network effects are acknowledged but not formally modeled.** The FORK test requires that reproduction be economically feasible, but network effects can make formally forkable systems practically unforkable. Signal's code is fully open; its network is not federated. A Signal fork without users is not a meaningful check on Signal Foundation governance. The framework flags this but does not provide a formal treatment of network-effect capture.

9.2 Future Work

Several extensions warrant investigation:

1. **Empirical measurement of correction velocity.** The velocity inequality (Section 5.1) predicts system instability when correction latency exceeds error accumulation rate. Historical case studies (PDS exclusion incidents, payment system outages, identity fraud waves) could quantify the threshold at which systems become ungovernable.
2. **International comparative analysis.** This paper focuses substantially on India Stack. Systematic comparison with Estonian X-Road, Singapore's National Digital Identity, the EU Digital Identity Wallet, and Brazil's Pix would test the framework's generality across political systems.
3. **Economic modeling of transition costs.** What does it cost to make an incorrigible system corrigible? Infrastructure-specific cost modeling, particularly for systems with existing vendor lock-in, would inform policy prioritization.
4. **AI-specific verification protocols.** The CODE test for learned systems (Section 6.2) requires layered transparency across the training pipeline. Developing standardized, machine-verifiable disclosure formats would enable systematic AI corrigibility assessment.
5. **Legal doctrine integration.** Constitutional courts increasingly review digital infrastructure. How should such courts operationalize corrigibility? Integration with proportionality analysis, due process doctrine, and administrative law would extend the framework's applicability.
6. **Network effect formalization.** A formal model of network-effect capture, specifying when a technically forkable system becomes practically unforkable, would sharpen the FORK test's application to platform infrastructure.
7. **Agentic system requirements.** Section 8.4.1 sketches how AI agents require corrigible infrastructure. Specifying precise API contracts for agent-compatible infrastructure would translate corrigibility requirements into engineering specifications.

Acknowledgments

This framework was developed through analysis of India Stack and subsequent generalization to universal criteria. The author thanks the digital rights community for ongoing critique and refinement.

License

This work is released under CC0 1.0 Universal (Public Domain). Use, adapt, translate, and redistribute freely. No attribution required.

Data and Code Availability

The complete framework, including JSON Schemas for machine-readable disclosure and assessment, is available at:

- Framework documentation: https://indiastack.in/dpi/
- Infrastructure manifest schema: https://indiastack.in/dpi/schema/infrastructure.json

- Corrigibility assessment schema: https://indiastack.in/dpi/schema/corrigibility.json
- Paper source and repository: https://github.com/AnivarAravind/corrigibility-framework

A Architecture of Accountability

To move beyond qualitative debate, this framework establishes a formal logic for accountability. The machine-readable artifacts described below serve not merely as data formats, but as a *digital constitutional binding* that separates operator claims from auditor verification. By enforcing specific data structures, we convert policy rhetoric into falsifiable assertions of system state.

A.1 Logic: The Adversarial Manifests

The architecture relies on a separation of concerns mirroring the adversarial legal process. It employs two distinct document types (see Figure 6):

1. **infrastructure.json (The Operator's Affidavit):** A signed declaration by the system operator asserting specific operational properties, governance commitments, and functional limitations.
2. **corrigibility.json (The Auditor's Finding):** An independent, cryptographically signed evaluation that tests the operator's claims against observable reality.

This architecture makes contradictions machine-readable. If an operator claims a governance mechanism is "binding" in their affidavit, but the auditor's observation marks the `user_authority` field as "advisory," the mismatch serves as mathematically verifiable proof of governance failure.

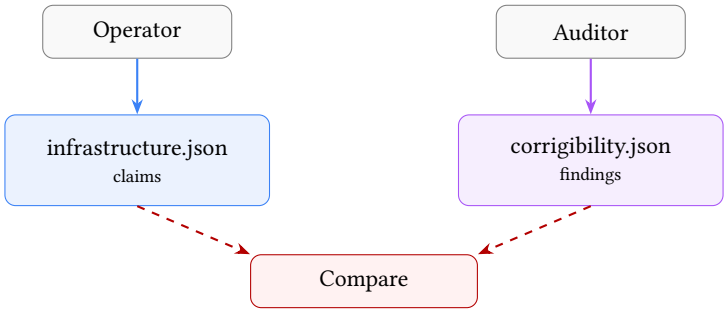

Figure 6: Adversarial Verification Architecture. Operators declare; Auditors verify. The structural gap constitutes the audit finding.

A.2 Interpretation: The Non-Authoritativeness of Determination

A critical logic gate in this framework is the derivation of status.

Claim 6. *The `determination` object in `corrigibility.json` is a derived convenience field. It is* ***not authoritative****.*

Any value asserted in `determination` MUST be mechanically recomputed from the `tests` object by evaluators or courts. If an auditor asserts `corrigible: true` but the `tests` object contains a failure, the manifest is strictly invalid. This rule ensures that no actor—operator, auditor, or regulator—can "declare" a system corrigible by fiat; status exists only as the computed sum of observable structural properties.

B Formal Structural Proofs

To formalize the assertion that "partial corrigibility is a fatal structural failure," we model Digital Public Infrastructure as a standard feedback control system subject to environmental disturbance.

B.1 Proof of Open-Loop Instability (The "Fatal Failure" Theorem)

Let a DPI system state $y(t)$ (actual impact) be expected to track a reference target $r(t)$ (intended policy/rights), subject to environmental disturbance $d(t)$ (attacks, changing demographics, fraud).

The error signal (grievance) is defined as:

$$e(t) = r(t) - y(t) \tag{8}$$

In a closed-loop system, the corrective action $u(t)$ (GOVERN) is a function of the detected error via a feedback controller K (GOVERN logic) and a sensor transfer function H (AUDIT/EXIT mechanism):

$$u(t) = K(t) \cdot H(e(t)) \tag{9}$$

The plant dynamics (DPI operation) are given by:

$$\dot{y}(t) = u(t) + d(t) \tag{10}$$

Theorem 1 (Null-Feedback Instability)**.** *If any component of the feedback path is structurally broken—specifically if Sensor Transparency $H = 0$ (Failed AUDIT) or Actuator Coupling $K = 0$ (Failed GOVERN)—the system effectively operates as Open Loop. Under Open Loop conditions with non-zero integrating disturbance $d(t)$, the error $e(t)$ diverges unboundedly.*

Proof. If tests **AUDIT** or **EXIT** fail, $H = 0$. If test **GOVERN** fails, $K = 0$. In either case, the effective corrective feedback is $u_{fb} = 0$. The system evolution depends only on initial intent u_{open} and disturbance:

$$y(t) = \int_0^t (u_{open}(\tau) + d(\tau))d\tau \tag{11}$$

Given that environmental variety $d(t)$ is stochastic and non-zero (Ashby's Law), and without negative feedback to compensate:

$$\lim_{t\to\infty} |r(t) - y(t)| \to \infty \tag{12}$$

Thus, stability is impossible without the closure of the feedback loop. "Partial" corrigibility (where $K = 0$ or $H = 0$) yields the same asymptotic stability profile as total authoritarianism. □

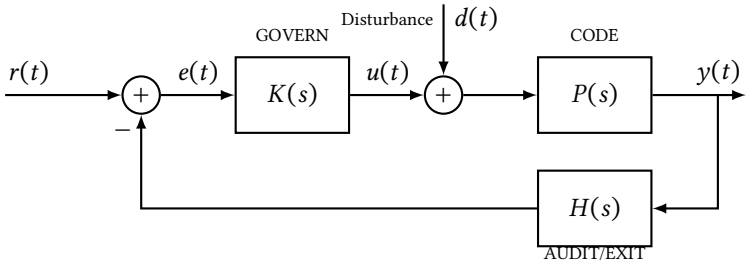

Figure 7: **Control-theoretic model of corrigible infrastructure.** $K(s)$: governance controller. $H(s)$: sensor/audit function. $P(s)$: plant (infrastructure). $d(t)$: environmental disturbance.

The closed-loop transfer function from reference to output is:

$$\frac{Y(s)}{R(s)} = \frac{K(s)P(s)}{1 + K(s)P(s)H(s)} \tag{13}$$

The transfer function from disturbance to output is:

$$\frac{Y(s)}{D(s)} = \frac{P(s)}{1 + K(s)P(s)H(s)} \tag{14}$$

Corrigibility requires that each component exists and is non-degenerate:

- EXIT $\Rightarrow e(t) \neq 0$ observable (error signal exists)
- AUDIT $\Rightarrow H(s) \neq 0$ (sensor functional)
- GOVERN $\Rightarrow K(s) \neq 0$ (actuator functional)
- CODE $\Rightarrow P(s)$ is intelligible
- FORK $\Rightarrow P, K, H$ replaceable if convergence fails

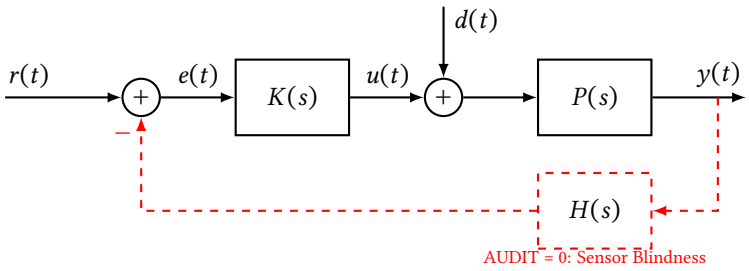

Figure 8: **Open-loop failure mode.** When AUDIT fails ($H = 0$), the feedback path is severed. Equivalent failure occurs if GOVERN ($K = 0$) or EXIT ($e(t) \equiv 0$) is removed.

B.2 The Strict Correction Velocity Inequality

For the system to remain stable not just asymptotically but locally (preventing irreversible harm), the correction rate must dominate the error rate. We formalize this with a friction coefficient.

Condition 1 (Strict Dynamic Stability)**.** *Let $\epsilon > 0$ represent the "friction cost" of correction (e.g., litigation time, protest cost). A system is stable iff:*

$$\left|\frac{d}{dt}C(t)\right| \geq \left|\frac{d}{dt}E(t)\right| + \epsilon \tag{15}$$

Implication: Post-hoc remedies (Courts/Ombudsmen) have a linear or constant rate of operation ($\frac{dC}{dt} = k$). Digital errors propagate at exponential or high-frequency rates ($\frac{dE}{dt} \propto e^t$). Since there is no t for which a linear function dominates an exponential one, relying on external courts guarantees eventual system collapse. The correction logic must be intrinsic to the loop.

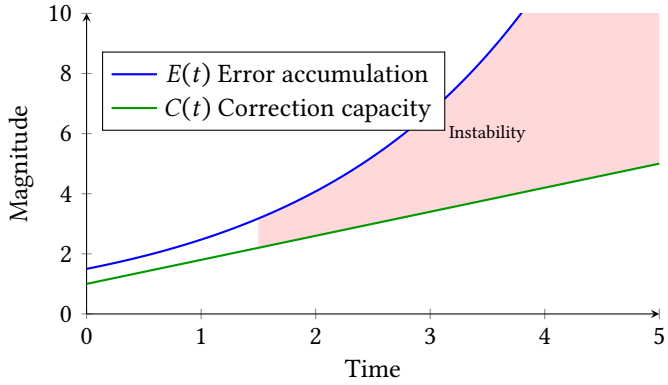

Figure 9: **Correction velocity mismatch.** Error $E(t)$ accumulates faster than correction $C(t)$ in bureaucratic systems. The shaded region represents accumulated unresolved harm.

B.3 Mapping to the Five Tests

Table 10: Mapping corrigibility tests to control-theoretic components

Test	Control Component	Failure Mode
EXIT	Error Signal	Reference collapse
CODE	Signal Integrity	Noise indistinguishable from intent
AUDIT	Sensor	Blind control
GOVERN	Actuator	No corrective force
FORK	Variety Expansion	Monolithic collapse

B.4 Agentic Interoperability Requirements

For an autonomous agent $\mathcal{A}$ to operate reliably within infrastructure $\mathcal{I}$, the following predicates must be strictly True:

1. **Reversibility:** $\forall$ state $s_t \in \mathcal{S}$, $\exists$ transition s_{t+1} (via EXIT) such that $\text{Cost}(s_{t+1}) < \infty$. (No deadlocks).
2. **Decidability:** $\forall$ input x, $\text{Compute}(x) \to \{0, 1\}$ is inspectable via **CODE**. (No hidden constraints).

Failure of these predicates renders $\mathcal{I}$ a "Hazmat Environment" for automated agents, effectively blocking the agentic economy.

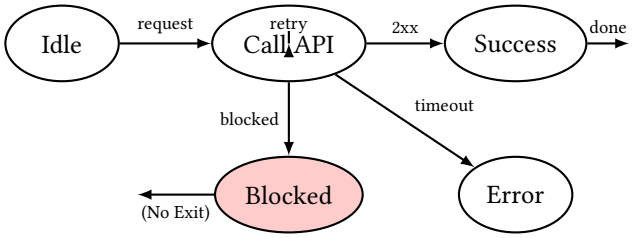

Figure 10: **Agentic automaton.** Without EXIT and observable error codes, agent loops indefinitely, times out, or escalates incorrectly.

B.5 Operational Proxies and Thresholds

Table 11: Operational proxies for the five tests. Thresholds are illustrative.

Test	Proxy Metric	Illustrative Threshold
EXIT	Penalty ratio p	$p \leq 1.2$
CODE	Visibility fraction v	$v \geq 0.9$
AUDIT	Mean time to detect (MTTD)	MTTD $\leq$ 24 hours
GOVERN	Bindingness + throughput τ	binding = true, $\tau \geq 1$ per 10k users/day
FORK	Resource feasibility ratio r	$r \leq 10^{-3}$

Interpretation:

- Penalty ratio p = cost(non-digital alternative) / cost(digital).
- Visibility v = public LOC / total LOC on critical execution path.
- Resource ratio r = estimated FLOP to retrain / aggregate community compute capacity.

B.6 Network Adoption Model for Fork Feasibility

Model adoption fraction $\alpha(t)$. A community-spawned fork succeeds if α crosses critical mass α^*. Logistic adoption with social influence:

$$\dot{\alpha} = \beta\alpha(1 - \alpha) + s(t) \tag{16}$$

where $s(t)$ is exogenous seeding. Fork feasibility requires parameters such that $\exists t$ with $\alpha(t) \geq \alpha^*$. Practical condition:

$$s_{\min} \geq \alpha^* - \alpha_0 \tag{17}$$

or subsidies that accelerate adoption rate β.

B.7 Summary of Formal Extensions

1. Linearized result ties removal of H, K, or EXIT to vanishing loop gain $L(s)$, exposing unstable process modes.
2. Integrator example supplies simple, indisputable divergence demonstration.
3. Correction velocity formalized via measurable $C(t)$ and $E(t)$.
4. Network model clarifies why legal forkability may fail in practice and yields policy levers (seeding, subsidies).
5. Agent automaton demonstrates need for explicit machine-facing EXIT/CODE/AUDIT primitives.

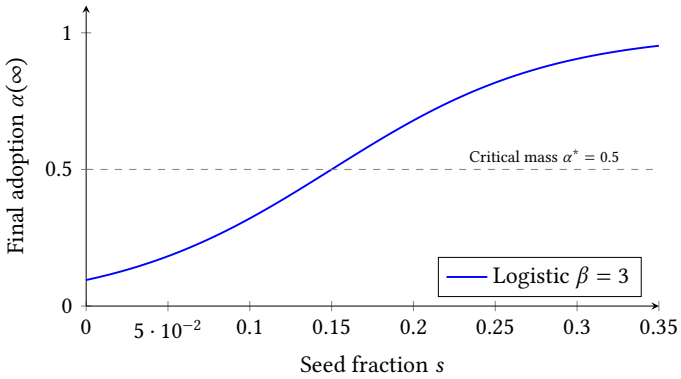

Figure 11: **Fork feasibility curve.** Below critical seeding, adoption does not reach critical mass. Parameters illustrative.

C Schema Specifications (v1.7)

The current schemas are normative. Implementations must validate against these structures to generate valid proofs of corrigibility.

C.1 Infrastructure Manifest (Operator Disclosure)

This schema forces operators to make definitive commitments. Note the specific inclusion of `offline_equivalent` to track the **EXIT** test and `when_uncertain` to track AI safety failures.

Listing 1: Operator Disclosure Schema (condensed)

```
{
  "$id": "https://indiastack.in/dpi/schema/infrastructure.json",
  "title": "DPI Operational Manifest v1.7",
  "required": ["meta", "access", "governance"],
  "properties": {
    "meta": {
      "required": ["system_id", "deployment", "lifecycle"],
      "properties": {
        "system_id": { "type": "string" },
        "deployment": { "enum": ["production","staging","pilot"] },
        "lifecycle": { "enum": ["active","deprecated","archived"] }
      }
    },
    "governance": {
      "required": ["model", "escalation_paths"],
      "properties": {
        "model": { "enum": ["community","foundation","corporate",
                            "government","closed"] },
        "escalation_paths": [{
          "required": ["level","url","binding"],
          "binding": { "type": "boolean" }
        }]
      }
    },
    "learned_components": [{
      "role": { "enum": ["advisory","decision","enforcement"] },
      "when_uncertain": { "enum": ["escalates","fails_open",
                                   "fails_closed","silent"] }
    }],
    "access": {
      "required": ["offline_equivalent"],
      "offline_equivalent": { "type": "boolean" }
    }
  }
}
```

C.2 Corrigibility Assessment (Auditor Assessment)

The auditor schema replaces subjective grading with hard observational measurements. Note the `penalty_ratio`, which quantifies the coerciveness of the system (the cost difference between digital and manual access).

Listing 2: Auditor Assessment Schema (condensed)

```
{
  "$id": "https://indiastack.in/dpi/schema/corrigibility.json",
  "title": "Corrigibility Assessment v1.7",
  "required": ["target","assessed_by","assessed_at","tests","determination"],
  "properties": {
    "target": { "type": "string" },
    "assessed_by": { "type": "string" },
    "assessed_at": { "format": "date" },
    "tests": {
      "required": ["exit","code","audit","govern","fork"],
      "exit": { "pass": "boolean", "penalty_ratio": "number" },
      "code": { "pass": "boolean",
                "visibility": { "enum": ["full","partial","none"] } },
      "audit": { "pass": "boolean", "fix_latency_observed": "string" },
      "govern": { "pass": "boolean",
                  "user_authority": { "enum": ["binding","veto",
                                               "advisory","none"] } },
      "fork": { "pass": "boolean", "barriers": ["string"] }
    },
    "determination": {
      "description": "Derived outcome. MUST be recomputed by evaluators.",
      "required": ["corrigible","tests_passed"],
      "corrigible": { "type": "boolean",
                      "description": "TRUE iff tests_passed == 5" },
      "tests_passed": { "type": "integer", "minimum": 0, "maximum": 5 }
    }
  }
}
```

D Verification and Enforcement

To distinguish valid governance from performative compliance, the framework enforces strict evidentiary rules for Identity and Authority.

D.1 Normative Rules

Rule A.9: Operational Definition of Binding Authority To satisfy the GOVERN test, any escalation path claimed as "binding" in the Manifest MUST include: (i) a resolvable reference to the statute or charter (`binding_proof_url`), (ii) a cryptographic hash of that instrument to prevent silent amendment, and (iii) historical evidence of enforcement against the operator. Assertions of authority that rely on post-execution bureaucratic discretion (e.g., ombudsmen, "feedback forms") DO NOT satisfy this rule and must be marked `false`.

Rule A.10: Chain of Custody Auditor manifests must be canonically serialized (RFC 8785), hashed (SHA-256), and signed (Ed25519) using a valid W3C Decentralized Identifier. Unsigned assessments are treated as schema-invalid. This creates a permanent, falsifiable record of who certified a system.

D.2 Identity Assurance Model

To prevent "Identity Washing"—where operators use ephemeral keys to rubber-stamp their own systems—the framework employs a Tiered Assurance Model. Verification dashboards should weigh assessments accordingly:

1. **Tier 1: Self-Attested (`did:key`):** Ephemeral identity. Suitable for whistleblowers, individual researchers, or rapid-response checks. *Status: Informational.*
2. **Tier 2: Domain-Verified (`did:web`):** Identity cryptographically bound to a specific DNS domain (e.g., `did:web:eff.org`). Attribution to known civil society actors. *Status: Verified.*
3. **Tier 3: Institution-Verified:** Identity anchored in transparency logs or institutional trust registries. *Status: Regulatory Standing.*

E Framework Evolution

This framework did not evolve through theoretical speculation; it hardened through adversarial engagement with live systems. Each version update represents the closure of a specific "loophole" used by operators to claim public legitimacy while retaining private control.

Version 1.0 (Definition)
Established the baseline thesis: Corrigibility is a structural precondition, distinct from feature sets. It defined the five tests to answer the foundational question: *What distinguishes public infrastructure from private monopoly?*

Version 1.2 (Assessment)
Closed the Self-Attestation Loophole. Early trials showed operators self-declaring success. This version separated the *Infrastructure Manifest* from the *Corrigibility Assessment*, establishing the adversarial verification model where publicness must be proven by independent parties.

Version 1.4 (Logic)
Closed the Ambiguity Loophole. Eliminated partial credit and "maturity models." The shift to strictly binary logic established that failure in any single dimension breaks the feedback loop required by Ashby's Law, rendering the system unstable.

Version 1.5 (Invariance)
Closed the Exceptionalism Loophole. Extended the framework to AI and decentralized ledgers, demonstrating that neither statistical opacity nor cryptographic immutability exempts a system from the requirements of Reversibility (EXIT) and Correction (GOVERN).

Version 1.6 (Constraint)
Closed the Consultation Loophole. Redefined GOVERN as non-overrideable internal constraint rather than advice or redress. Explicitly disqualified post-execution remedies and advisory bodies from constituting system governance.

Version 1.6.2 (Reproduction)
Closed the Substitution Loophole. Distinguished EXIT (market choice) from FORK (reproduction). Switching landlords is not forking infrastructure.

Version 1.7 (Layering)
Closed the DID-Washing Loophole. Added layer-decomposed evaluation requiring analysis at each architectural stratum. Systems claiming corrigibility through identifier decentralization while retaining centralized governance at higher layers are now explicitly diagnosed. Introduced the propagation rule: $T_{\text{system}} = \bigwedge T(L_i)$.

References

Ashby, W. Ross. *An Introduction to Cybernetics.* Chapman & Hall, 1956.

Ostrom, Elinor. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990.

Stallman, Richard. *Free Software, Free Society: Selected Essays.* GNU Press, 2002.

Winner, Langdon. Do Artifacts Have Politics? *Daedalus*, 109(1):121–136, 1980.

Hirschman, Albert O. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States.* Harvard University Press, 1970.

Lessig, Lawrence. *Code: Version 2.0.* Basic Books, 2006.

Bowker, Geoffrey C. and Star, Susan Leigh. *Sorting Things Out: Classification and Its Consequences.* MIT Press, 1999.

G20. New Delhi Leaders' Declaration. G20 Summit, September 2023.

United Nations Development Programme. Digital Public Infrastructure: Definitions and Core Concepts. UNDP Digital Strategy Office, 2024.

Gebru, Timnit, Morgenstern, Jamie, Vecchione, Briana, et al. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Mitchell, Margaret, Wu, Simone, Zaldivar, Andrew, et al. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

Bhatia, Gautam. *The Transformative Constitution: A Radical Biography in Nine Acts.* HarperCollins India, 2019.

Leveson, Nancy G. *Engineering a Safer World: Systems Thinking Applied to Safety.* MIT Press, 2011.

Khera, Reetika. *Dissent on Aadhaar: Big Data Meets Big Brother.* Orient BlackSwan, 2019.

Cohen, Julie E. *Between Truth and Power: The Legal Constructions of Informational Capitalism.* Oxford University Press, 2019.

Supreme Court of India. Justice K.S. Puttaswamy (Retd.) v. Union of India. Writ Petition (Civil) No. 494 of 2012, (2017) 10 SCC 1.

Collingridge, David. *The Social Control of Technology*. Frances Pinter, 1980.

Pasquale, Frank. *The Black Box Society: The Secret Algorithms That Control Money and Information*. Harvard University Press, 2015.

World Wide Web Consortium. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, July 19, 2022. https://www.w3.org/TR/did-core/

European Union. Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 (eIDAS 2.0). *Official Journal of the European Union*, 2024.

Khera, Reetika. Impact of Aadhaar on Welfare Programmes. *Economic and Political Weekly*, 52(50):61–70, 2017.

European Commission. Study about the Impact of Open Source Software and Hardware on Technological Independence, Competitiveness and Innovation in the EU Economy. European Commission, Directorate-General for Communications Networks, Content and Technology, 2021.